Designing fraud-prevention systems that keep analysts in the loop

José P. Barrantes

IBM · Programa de Posgrado en Computación e Informática, Universidad de Costa Rica

jospablo777@gmail.com · linkedin.com/in/jose-barrantes

Companion article for the Nerdearla Chile 2026 talk · Santiago, April 17, 2026

Abstract. This article extends the Nerdearla Chile 2026 talk on fraud prevention, machine learning, and design patterns. The main claim is that fraud prevention should be treated as an **ML-enabled software-architecture problem**, not only as a classification problem. The article develops a design-pattern proposition for **ML-enabled Human-in-the-Loop (HIL) Triage**: rules for explicit cases, ML for scoring, policy for routing, and expert analysts for ambiguous cases and structured feedback. The discussion connects this architecture to the literature on software architecture for ML systems, human-in-the-loop design, design patterns for AI-based systems, learning to defer, alert prioritization, reliable ML, and platform engineering (Mosqueira-Rey et al., 2023).

1 Introduction

This article fills the gap left by a short conference talk: the broader software-architecture background, the design-pattern framing, the operational consequences, and the connections to MLOps and platform engineering.

The central thesis: robust fraud-prevention systems are hybrid systems. They rarely succeed as purely manual or purely automatic workflows. The most resilient architecture combines deterministic controls, statistical scoring, decision policy, and expert human judgment.

Hybrid system: Combines deterministic rules, ML scoring, decision policy, and expert human judgment

That claim is architectural. Production ML systems require supporting software, data pipelines, monitoring, deployment structures, and cross-functional engineering practices (Amershi et al., 2019a; Lewis et al., 2021b; Nazir et al., 2024; Sculley et al., 2015). In fraud prevention, the surrounding system also includes analysts, decision queues, escalation paths, and operational feedback loops. The unit of design is the socio-technical system (Dobbe and Wolters, 2024; Salwei and Carayon, 2022).

Socio-technical system: Behavior shaped by both technical components (models, pipelines, infrastructure) and human work (analyst review, organizational decisions, governance).

Figure 1. Effective fraud systems are built from the interaction of rules, ML, and expert analysts.

Figure 1 captures the argument in its simplest form. Rules encode explicit knowledge. ML compresses signals into scores. Analysts contribute judgment, novelty detection, and feedback. The point is to design their interaction well.

2 Why fraud prevention becomes an architecture problem

Fraud prevention is not a stable prediction problem. Systems combine rules, supervised models, anomaly detection, and human investigation because no single technique handles novelty, delayed labels, and operational constraints (Carcillo et al., 2021; Hernandez Aros et al., 2024).

Adversarial adaptation: Fraudsters change tactics once a

First, attackers adapt (Bolton and Hand, 2002; Carcillo et al., 2021; Lunghi et al., 2023).

Label delay: Ground truth is often confirmed only after complaints or investigations—

Second, labels are delayed and imperfect (Carcillo et al., 2021; Lunghi et al., 2023). The system acts inside the environment that later generates its own labels.

Asymmetric error costs: False negatives cause direct loss; false

Third, error costs are asymmetric (Bolton and Hand, 2002; Hernandez Aros et al., 2024).

Time sensitivity: A correct decision made too late may still be operationally poor.

Fourth, many decisions are time-sensitive (Bolton and Hand, 2002; Carcillo et al., 2021; Lunghi et al., 2023). Architecture determines which cases are handled automatically and how scarce analyst attention is reserved for the ambiguous zone.

These properties explain the move toward system-centric and architecture-centric thinking (Lewis et al., 2021b; Muccini and Vaidhyanathan, 2021; Nazir et al., 2024).

3 From hidden technical debt to ML-enabled fraud systems

Hidden technical debt: The real cost of production ML lies mostly outside the model: data pipelines, configuration, serving, monitoring, and glue code.

Sculley et al. (2015) argued that much of the real cost of ML systems lies outside the model itself. A fraud model is never the whole fraud system.

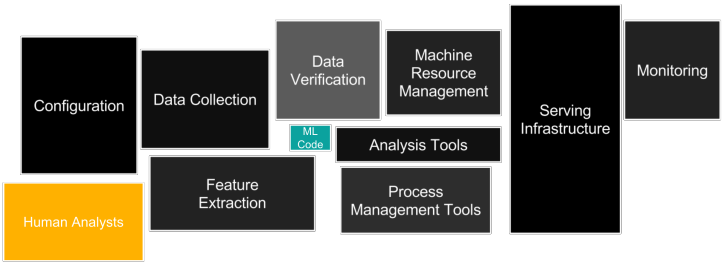

Figure 2. System anatomy for fraud operations. Adapted from Sculley et al. (2015).

Figure 2 places the model inside a wider environment. Lin and Ryaboy (2013) reinforce this from large-scale data-mining infrastructure.

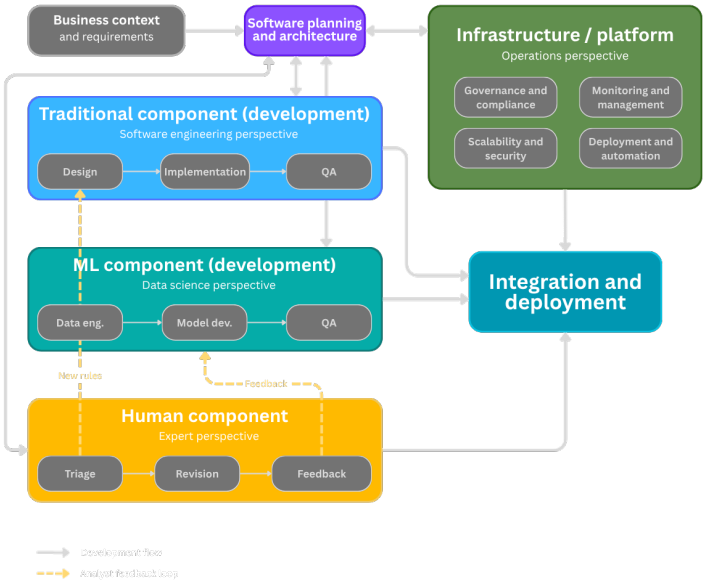

Figure 3. Planning view. Adapted from Lewis et al. (2021b), Andersen and Maalej (2024), and Kästner (2025).

Co-versioning: Data, features, models, evaluation artifacts, and policy thresholds should be traceable together.

Figure 3 broadens the lens to planning. Lewis et al. (2021b) motivate co-architecting and co-versioning. The added human lane makes explicit what fraud operations often leave implicit.

4 Why this article proposes a design pattern

Design pattern: A reusable solution to a recurring problem.

The architecture is proposed as a reusable response to a recurring class of problems.

ML architecture patterns: Washizaki et al. identified 33 ML patterns. Heiland et al. expanded to 70.

Washizaki et al. (2020) documented recurring ML patterns. Heiland et al. (2023) expanded the repository. Järvenpää et al. (2024) focused on reusable tactics. Cruz et al. (2023) reinforced architecture rationale.

HIL patterns: Andersen & Maalej catalog reusable HIL

Andersen and Maalej (2024) frame HIL as a design space with reusable patterns.

HITL-ML family: Active learning, interactive ML, machine teaching. Differs in *how* and *when* human expertise enters the loop.

Mosqueira-Rey et al. (2023) describe HITL-ML as a family of approaches.

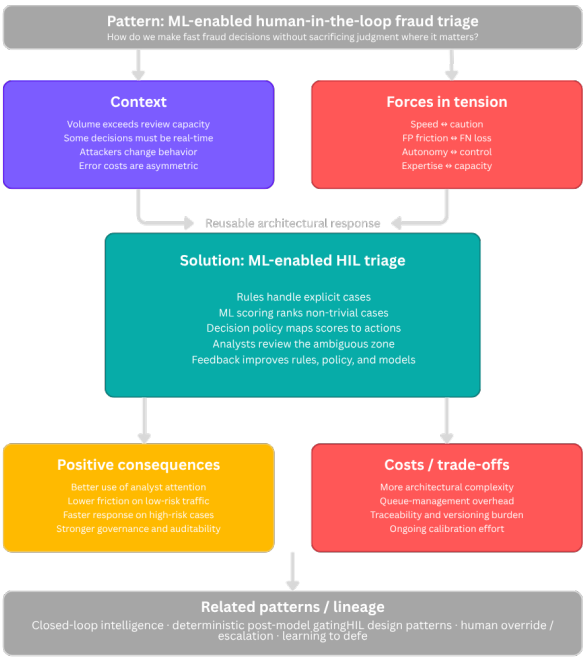

Figure 4. Pattern framing for ML-enabled HIL fraud triage. Synthesized from Lakshmanan et al. (2020), Andersen and Maalej (2024), Heiland et al. (2023), Cruz et al. (2023), Järvenpää et al. (2024).

Figure 4 summarizes the pattern view. Forces and consequences make trade-offs explicit, which is central to architectural reasoning (Richards and Ford, 2025).

5 The anatomy of the proposed pattern

The pattern is a series of transformations: observation to score, score to policy, policy to action.

Score ≠ decision: The model produces a risk estimate. A separate policy layer translates it into action under business constraints.

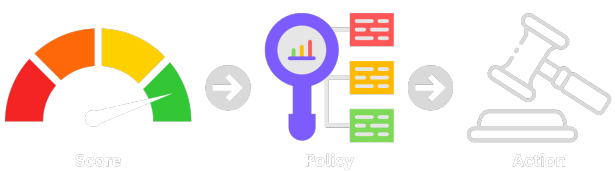

Figure 5. Score-to-policy-to-action pipeline.

The separation in Figure 5 avoids treating the score as a business decision (Chen et al., 2022; Kästner, 2025).

Learning to defer (L2D): The system decides whether to handle a case automatically or route it to a human expert.

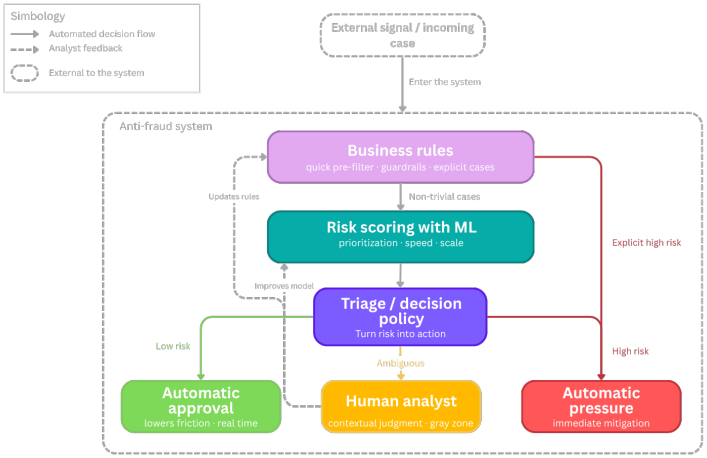

Figure 6. Reference architecture of the ML-enabled HIL Triage pattern.

Figure 6 summarizes the runtime logic. This aligns with the learning-to-defer literature (Alves et al., 2025).

6 Analysts are a runtime component, not a fallback afterthought

Analyst as component: Designed workflow with structured inputs (ranked cases, reason codes) and structured outputs (decisions, feedback).

Analysts should be explicit runtime components.

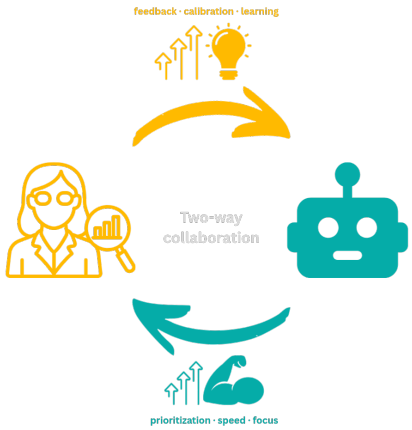

Figure 7. Bidirectional collaboration between analysts and ML.

Figure 7 shows two-way collaboration (Ding et al., 2023; Mosqueira-Rey et al., 2023).

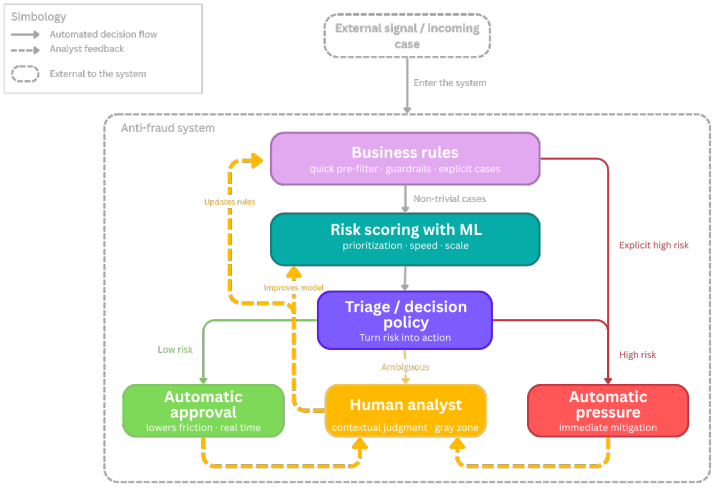

Figure 8. Closed-loop HIL triage with explicit feedback paths.

Kadam (2024) treats feedback as propagable and reusable signal.

Expert-in-the-loop approaches suggest human review is most valuable where the model faces novelty (Yuan et al., 2026).

Analysts should also audit automatically resolved cases (Andersen and Maalej, 2024; Chen et al., 2022; Kadam, 2024; Tan Wei Hao et al., 2026).

Feedback propagation: Even sparse analyst

Open-set recognition: Detecting cases outside any known class.

Retrospective audit: Review samples of automatically resolved cases to catch errors and revise routing logic.

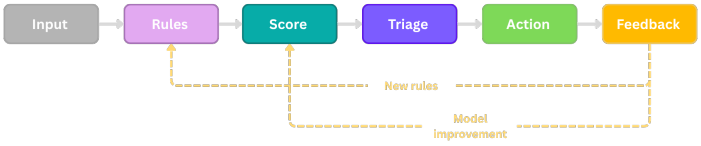

Figure 9. Operative learning: the pattern is a loop, not a pipeline.

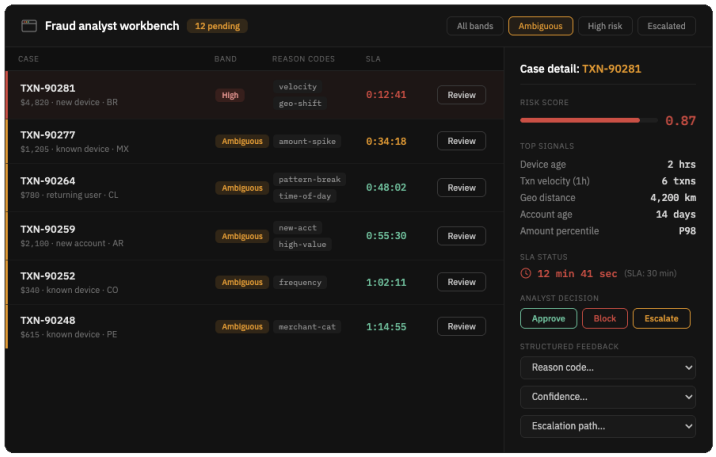

Figure 10. Analyst workbench. Synthesized from Jalalvand et al. (2024), Ghadermazi et al. (2024), Alves et al. (2025).

Analyst workbench: Ranked cases, risk bands, reason codes, SLA timers, action controls, structured feedback fields.

Figure 10 turns the architecture into an operational review system (Alves et al., 2025; Amershi et al., 2019b; Ghadermazi et al., 2024; Jalalvand et al., 2024; Zelvenskiy et al., 2022).

7 Evaluation must fit the socio-technical system

Evaluation has to widen (Dobbe and Wolters, 2024; Lewis et al., 2021b; Nazir et al., 2024; Salwei and Carayon, 2022).

Four evaluation layers: (1) Data/input quality, (2) model/decision quality, (3) queue/analyst operations, (4) runtime/platform quality.

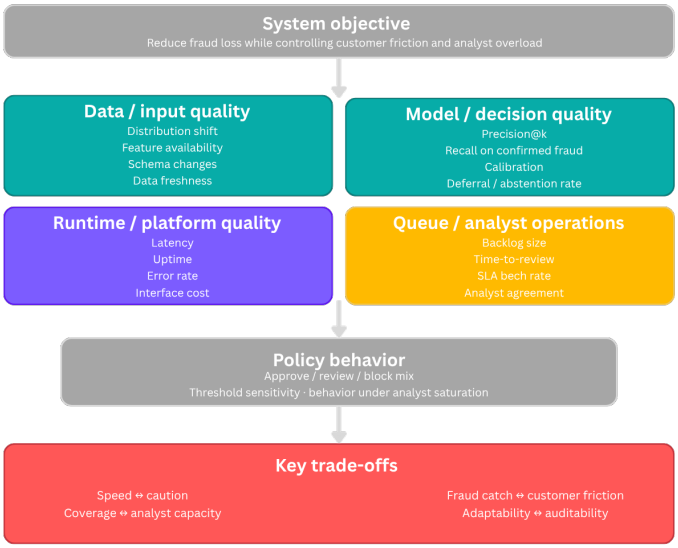

Figure 11. Metrics and trade-offs for ML-enabled HIL fraud triage.

Figure 11 defines a broader evaluation frame (Alves et al., 2025; Chen et al., 2022; Dobbe and Wolters, 2024; Fazelpour, 2025; Ghadermazi et al., 2024; Jalalvand et al., 2024; Lewis et al., 2021b; Tan Wei Hao et al., 2026).

Deferral rate: Fraction of cases routed to human review. Key tuning parameter for

The policy box is equally important (Chen et al., 2022; Kästner, 2025; Lewis et al., 2021b). The trade-off strip reflects operational frontiers (Chen et al., 2022; Fazelpour, 2025; Richards and Ford, 2025).

Policy behavior: The approve/review/block split is an organizational choice, not just a model output.

8 MLOps and platform engineering are part of the argument

The pattern has an operational lifecycle (Kästner, 2025; Lewis et al., 2021b; Tan Wei Hao et al., 2026).

MLOps lifecycle: Data → features → training → eval → deploy → monitor → review → feedback. Three return loops.

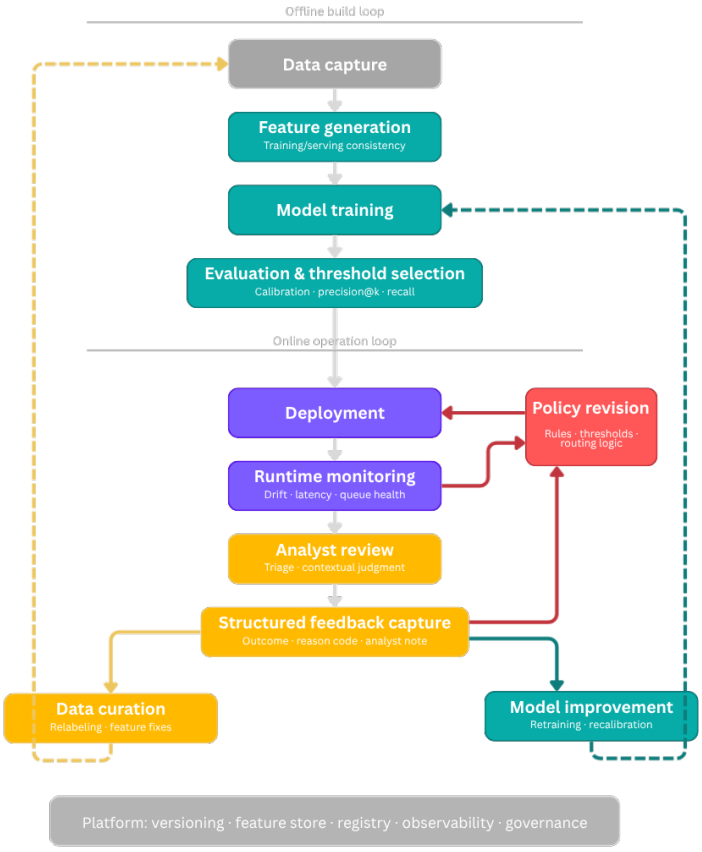

Figure 12. MLOps feedback lifecycle. Adapted from Lewis et al. (2021b), Tan Wei Hao et al. (2026), Kadam (2024).

Monitorability: First-class quality attribute. Separate monitoring for data,

ML platform: Versioning, feature store, model registry, observability, governance. Reduces accidental complexity.

The side loops show why feedback should not be collapsed (Kadam, 2024; Konieczny, 2025; Lewis et al., 2021a,b; Lin and Ryaboy, 2013; Mosqueira-Rey et al., 2023).

Versioning, feature storage, registry services, observability, and governance are part of what makes the architecture maintainable.

9 What the pattern improves, and what it costs

The pattern improves analyst efficiency (Alves et al., 2025; Jalalvand et al., 2024), reduces friction (Jalalvand et al., 2024; Kästner, 2025), separates inference from governance (Chen et al., 2022; Lewis et al., 2021b), and makes learning explicit (Andersen and Maalej, 2024; Mosqueira-Rey et al., 2023). Costs include more moving parts (Lewis et al., 2021b; Nazir et al., 2024), queue monitoring (Ghadermazi et al., 2024; Jalalvand et al., 2024), versioned decision logic (Cruz et al., 2023; Lewis et al., 2021b), and cross-team coordination (Nazir et al., 2024; Richards and Ford, 2025).

9.1 Anti-patterns worth naming

Anti-pattern — Score = policy: Raw scores trigger actions; governance becomes opaque.

Letting the score become policy (Lewis et al., 2021b; Washizaki et al., 2020). Remedy: Section 5 (Kästner, 2025; Washizaki et al., 2020).

Anti-pattern — Unstructured HIL: Free-text-only review generates cost but no reusable signal.

Treating analysts as unstructured exception handling (Andersen and Maalej, 2024; Mosqueira-Rey et al., 2023).

Building the model while leaving operations underspecified (Jalalvand et al., 2024; Lewis et al., 2021b; Nazir et al., 2024).

Ignoring monitorability until something breaks (Jalalvand et al., 2024; Lewis et al., 2021b).

Glue code and pipeline jungles (Kästner, 2025; Lewis et al., 2021b; Washizaki et al., 2020). These anti-patterns define the pattern's limits (Cruz et al., 2023; Richards and Ford, 2025).

Anti-pattern — Pipeline jungle: Feature joins, rule engines, risk services accumulate without platform discipline.

10 Why this matters for a new, but important field

ML-enabled software architecture is still young (Muccini and Vaidhyanathan, 2021; Nazir et al., 2024). Human participation is itself a design space (Andersen and Maalej, 2024; Mosqueira-Rey et al., 2023). Fraud makes the socio-technical nature of ML visible (Dobbe and Wolters, 2024; Kästner, 2025; Salwei and Carayon, 2022). This pattern adds one concrete example to that growing field (Andersen and Maalej, 2024; Kästner, 2025; Richards and Ford, 2025; Washizaki et al., 2020).

11 Open questions

One concerns **policy optimization**. Another concerns **feedback quality** (Alves et al., 2025; Yuan et al., 2026). A third concerns **architecture evaluation** (Chen et al., 2022; Cruz et al., 2023; Jalalvand et al., 2024; Lunghi et al., 2023; Nazir et al., 2024). A fourth concerns **platform productization** (Lin and Ryaboy, 2013; Tan Wei Hao et al., 2026).

A fifth concerns **LLMs in the analyst workflow**: how to integrate case summarization, reasoning traces, and explanation interfaces without weakening governance (Tan Wei Hao et al., 2026).

LLM integration: How to add NL case summarization and conversational investigation without undermining structured feedback and auditability.

12 Conclusion

Fraud prevention should be designed as an ML-enabled socio-technical architecture. Use rules where explicit knowledge exists. Use ML where prioritization creates scale. Use analysts where context and judgment matter. Keep score separate from policy. Capture feedback in structured form. Design the platform so the system can be observed, revised, and improved.

This combination of rules, scoring, policy, analyst review, and closed-loop learning deserves to be treated as a reusable architectural response for a

high-stakes domain.

References

J. a. V. Alves, D. Leitão, S. Jesus, M. O. P. Sampaio, J. Liébana, P. Saleiro, M. A. T. Figueiredo, and P. Bizarro. A benchmarking framework and dataset for learning to defer in human-AI decision-making. *Scientific Data*, 12:506, 2025. doi: 10.1038/s41597-025-04664-y.

S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, and B. Nushi. Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 291–300. IEEE, 2019a. doi: 10.1109/ICSE-SEIP.2019.00042.

S. Amershi, D. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. T. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, and E. Horvitz. Guidelines for human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, 2019b. doi: 10.1145/3290605.3300233.

J. Andersen and W. Maalej. Design patterns for machine learning-based systems with humans in the loop. *IEEE Software*, 41(4):151–159, 2024. doi: 10.1109/MS.2023.3340256.

R. J. Bolton and D. J. Hand. Statistical fraud detection: A review. *Statistical Science*, 17(3):235–255, 2002. doi: 10.1214/ss/1042727940.

F. Carcillo, Y.-A. Le Borgne, O. Caelen, Y. Kessaci, F. Oblé, and G. Bontempi. Combining unsupervised and supervised learning in credit card fraud detection. *Information Sciences*, 557:317–331, 2021. doi: 10.1016/j.ins.2019.05.042.

C. Chen, N. R. Murphy, K. Parisa, D. Sculley, and T. Underwood. *Reliable Machine Learning: Applying SRE Principles to ML in Production*. O'Reilly Media, 2022.

P. Cruz, G. Ulloa, D. San Martin, and A. Veloz. Software architecture evaluation of a machine learning enabled system: A case study. In *2023 42nd IEEE International Conference of the Chilean Computer Science Society (SCCC)*. IEEE, 2023. doi: 10.1109/SCCC59417.2023.10315755.

X. Ding, N. Seleznev, S. Kumar, C. B. Bruss, and L. Akoglu. From detection to action: A human-in-the-loop toolkit for anomaly reasoning and management. In *Proceedings of the Fourth ACM International Conference on AI in Finance*, pages 279–287. Association for Computing Machinery, 2023. doi: 10.1145/3604237.3626872.

R. Dobbe and A. Wolters. Toward sociotechnical AI: Mapping vulnerabilities

for machine learning in context. *Minds and Machines*, 34(2):Article 12, 2024. doi: 10.1007/s11023-024-09668-y.

S. Fazelpour. Disciplining deliberation: A socio-technical perspective on machine learning trade-offs. *The British Journal for the Philosophy of Science*, 2025. doi: 10.1086/734552. Advance online publication.

J. Ghadermazi, A. Shah, and S. Jajodia. A machine learning and optimization framework for efficient alert management in a cybersecurity operations center. *Digital Threats: Research and Practice*, 5(2):Article 19, 2024. doi: 10.1145/3644393.

L. Heiland, M. Hauser, and J. Bogner. Design patterns for AI-based systems: A multivocal literature review and pattern repository. In *2023 IEEE/ACM 2nd International Conference on AI Engineering—Software Engineering for AI (CAIN)*. IEEE, 2023. doi: 10.1109/CAIN58948.2023.00034.

L. Hernandez Aros, L. X. Bustamante Molano, F. Gutierrez-Portela, J. J. Moreno Hernandez, and M. S. Rodríguez Barrero. Financial fraud detection through the application of machine learning techniques: A literature review. *Humanities and Social Sciences Communications*, 11(1):1–22, 2024. doi: 10.1057/s41599-024-03606-0.

F. Jalalvand, M. B. Chhetri, S. Nepal, and C. Paris. Alert prioritisation in security operations centres: A systematic survey on criteria and methods. *ACM Computing Surveys*, 57(2):Article 42, 2024. doi: 10.1145/3695462.

H. Järvenpää, P. Lago, J. Bogner, G. A. Lewis, H. Muccini, and I. Ozkaya. A synthesis of green architectural tactics for ML-enabled systems. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS '24)*, pages 130–141. Association for Computing Machinery, 2024. doi: 10.1145/3639475.3640111.

P. Kadam. Enhancing financial fraud detection with human-in-the-loop feedback and feedback propagation. In *2024 International Conference on Machine Learning and Applications (ICMLA)*, pages 1198–1203. IEEE, 2024. doi: 10.1109/ICMLA61862.2024.00185.

C. Kästner. *Machine Learning in Production: From Models to Products*. MIT Press, 2025. URL https://mlip-cmu.github.io/book/.

B. Konieczny. *Data Engineering Design Patterns: Recipes for Solving the Most Common Data Engineering Problems*. O'Reilly Media, 2025.

V. Lakshmanan, S. Robinson, and M. Munn. *Machine Learning Design Patterns*. O'Reilly Media, 2020.

G. A. Lewis, S. Bellomo, and I. Ozkaya. Characterizing and detecting mismatch in machine-learning-enabled systems. In *2021 IEEE/ACM 1st Workshop on AI Engineering—Software Engineering for AI (WAIN)*, pages 133–140. IEEE, 2021a. doi: 10.1109/WAIN52551.2021.00028.

G. A. Lewis, I. Ozkaya, and X. Xu. Software architecture challenges for ML systems. In *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 634–638. IEEE, 2021b. doi: 10.1109/ICSME52107.2021.00071.

J. Lin and D. Ryaboy. Scaling big data mining infrastructure: The Twitter experience. *ACM SIGKDD Explorations Newsletter*, 14(2):6–19, 2013. doi: 10.1145/2481244.2481247.

D. Lunghi, A. Simitsis, O. Caelen, and G. Bontempi. Adversarial learning in real-world fraud detection: Challenges and perspectives. In *Proceedings of the Second ACM Data Economy Workshop*, pages 27–33. Association for Computing Machinery, 2023. doi: 10.1145/3600046.3600051.

E. Mosqueira-Rey, E. Hernández-Pereira, D. Alonso-Ríos, J. Bobes-Bascarán, and A. Fernández-Leal. Human-in-the-loop machine learning: A state of the art. *Artificial Intelligence Review*, 56:3005–3054, 2023. doi: 10.1007/s10462-022-10246-w.

H. Muccini and K. Vaidhyanathan. Software architecture for ML-based systems: What exists and what lies ahead. In *2021 IEEE/ACM 1st Workshop on AI Engineering—Software Engineering for AI (WAIN)*, pages 121–128. IEEE, 2021. doi: 10.1109/WAIN52551.2021.00026.

R. Nazir, A. Bucaioni, and P. Pelliccione. Architecting ML-enabled systems: Challenges, best practices, and design decisions. *Journal of Systems and Software*, 207:111860, 2024. doi: 10.1016/j.jss.2023.111860.

M. Richards and N. Ford. *Fundamentals of Software Architecture: A Modern Engineering Approach*. O'Reilly Media, 2nd edition, 2025.

M. E. Salwei and P. Carayon. A sociotechnical systems framework for the application of artificial intelligence in health care delivery. *Journal of Cognitive Engineering and Decision Making*, 16(4):194–206, 2022. doi: 10.1177/15553434221097357.

D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

B. Tan Wei Hao, S. Padmanabhan, and V. Mallya. *Machine Learning Platform Engineering: Build an Internal Developer Platform for ML and AI Systems*. Manning, 2026.

H. Washizaki, H. Uchida, F. Khomh, and Y.-G. Guéhéneuc. Machine learning architecture and design patterns. *IEEE Software*, 37(4):76–84, 2020. doi: 10.1109/MS.2019.2961960.

X. Yuan, P. Yu, S. Liu, Z. Sun, Y. Zhang, and J. Xu. An expert-in-the-loop framework for unknown attack detection via open-set recognition. *Journal*

of Computer Security, 2026. doi: 10.1177/0926227X251414058. Advance online publication.

S. Zelvenskiy, G. Harisinghani, T. Yu, E. Ng, and R. Wei. Project RADAR: Intelligent early fraud detection system with humans in the loop. Uber Blog, 2022. URL https://www.uber.com/en-CR/blog/project-radar-intelligent-early-fraud-detection/.